

Dev.to API ETL Pipeline -Documentation

Project Summary

This project implements a production-style ETL pipeline using Prefect, PostgreSQL, SQLAlchemy, Pandas and HTTPX. The pipeline extracts articles from the Dev.to API, transforms JSON responses, loads them into PostgreSQL and performs watermark-based incremental loading.

Business Problem

Manual API ingestion leads to duplicate processing, lack of monitoring, poor traceability, and repeated full loads. The solution automates extraction, transformation, loading, and run auditing.

Tech Stack

Python, Prefect, PostgreSQL, SQLAlchemy ORM, Pandas, HTTPX.

Architecture

Dev.to API -> Extract (HTTPX) -> Transform (Pandas) -> Incremental Filter (Watermark) -> Load (SQLAlchemy ORM) -> PostgreSQL -> Audit Metadata Table.

Project Structure

database.py: Connection Management. models.py: ORM models. create_tables.py: schema creation. audit.py: ETL audit utilities. etlflow.py: orchestrates ETL.

Database Design - Articles

Stores article information including id, title, published_at, url, comments_count, positive_reactions_count, tag_list, username and load_timestamp.

```
prefect=# select * from articles limit 5;
```

id	url	title	published_at	comments_count	positive_reactions_count	tag_list
username	load_timestamp					
4036380	Forward Deployed Engineers and the future of software engineering	2026-07-01 05:50:18+05:30	https://dev.to/dailycontext/forward-deployed-engineers-and-the-future-of-software-engineering-ill	0	5	agents,ai,career,softw
areengineering	ricmac	2026-07-06 16:06:00.645645+05:30				
4055898	Protect Yourself, Mesh Yourself	2026-07-03 19:15:00+05:30	https://dev.to/eschmechel/pro	1	7	tutorial,productivity,
tect-yourself-mesh-yourself-3fkn						
opensource,devops	eschmechel	2026-07-06 16:06:00.667455+05:30				
4052404	Letting the DEV Community Weigh in on the Topics of AIE	2026-07-02 20:54:12+05:30	https://dev.to/dailycontext/l	8	64	aie,ai, discuss
etting-the-dev-community-weigh-in-on-the-topics-of-aie-439l	ben	2026-07-06 16:06:00.678724+05:30				
4052238	The Fragile Balance of AI Development: Individual Flow vs. Collective Context	2026-07-02 20:30:21+05:30	https://dev.to/dailycontext/t	0	12	aie,agents,mcp,devops
he-fragile-balance-of-ai-development-individual-flow-vs-collective-context-2f49	ben	2026-07-06 16:06:00.687279+05:30				
4015354	What's Next for AI?	2026-06-29 12:48:48+05:30	https://dev.to/sylwia-lask/wh	253	196	ai,llm,webdev
ats-next-for-ai-2191	sylwia-lask	2026-07-06 16:06:00.69794+05:30				

(5 rows)

Database Design - ETLMetadata

Stores run level audit information: pipeline_name, run_start_time, run_end_time, status, records_extracted, records_loaded, watermark and error_message.

```
prefect=# select * from etl_metadata;
```

id	pipeline_name	run_start_time	run_end_time	status	records_extracted	records_loaded	watermark	error_message
1	devto_etl	2026-07-07 14:53:47.889994+05:30	2026-07-07 14:53:56.028415+05:30	SUCCESS	90	90	2026-07-07 07:14:15+05:30	

Execution Flow

1. Create audit run record.
2. Fetch API pages.
3. Convert JSON to DataFrame.
4. Read the latest successful watermark.
5. Filter incremental data.
6. Load records into PostgreSQL.
7. Update audit table as SUCCESS or FAILED.
8. Store new watermark.

Incremental Loading Strategy

The latest successful watermark is retrieved from ETLMetadata. Only records whose published_at > latest successful watermark is processed. The next watermark is set to the maximum published_at value successfully loaded. This prevents reloading existing records.

Audit Logging Enhancement

A separate audit layer tracks every ETL execution. Each run captures run_start_time, run_end_time, status, records_extracted, records_loaded, watermark and error_message, enabling ETL monitoring and troubleshooting.

Validation Queries

Validate watermark alignment by comparing MAX(published_at) from the articles table with the latest successful watermark from ETLMetadata. Use COUNT(*), MIN(published_at) and MAX(published_at) to validate loads.

max_article_timestamp	latest_watermark
2026-07-07 07:14:15+05:30	2026-07-07 07:14:15+05:30
(1 row)	

#Check your incremental logic

SELECT

```
(
  SELECT MAX(published_at)
  FROM articles
) AS max_article_timestamp,

(
  SELECT watermark
  FROM etl_metadata
  WHERE pipeline_name = 'devto_etl'
  AND status = 'SUCCESS'
  ORDER BY id DESC
  LIMIT 1
) AS latest_watermark;
```

```
-----

SELECT

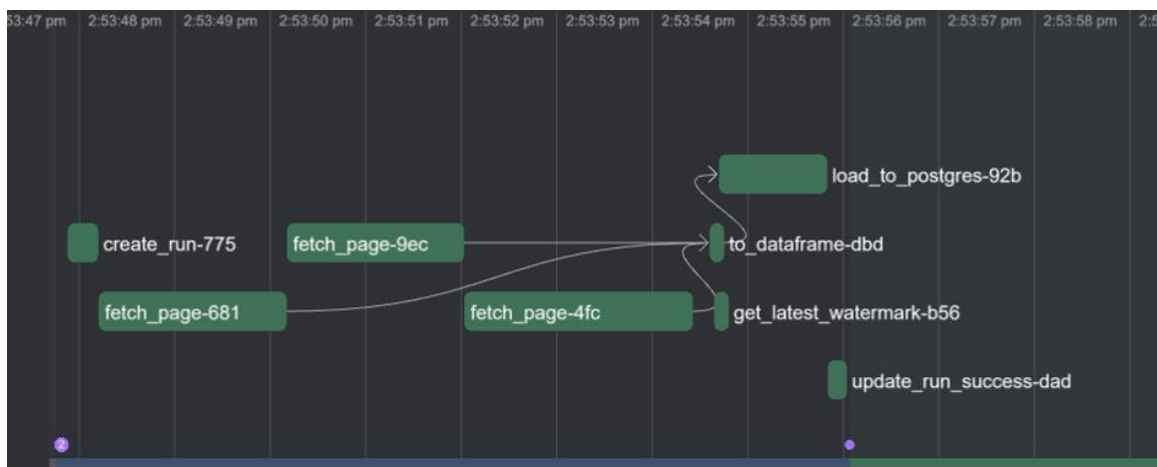
  COUNT(*),

  MIN(published_at),

  MAX(published_at)

FROM articles;
```

Prefect Orchestration



Prefect was used as the workflow orchestration framework for this ETL solution. The Prefect UI provides a graphical timeline view of task execution. The flow graph below shows task dependencies and execution ordering within a successful ETL run.

The execution timeline demonstrates the following sequence:

1. Audit record creation (create_run)
2. API extraction (fetch_page)
3. Data transformation (to_dataframe)
4. Watermark retrieval (get_latest_watermark)
5. PostgreSQL loading (load_to_postgres)
6. Audit table update (update_run_success)

Error Handling using Prefect: The extraction task implements retry behavior

<code>@task (retries=3, retry_delay_seconds=[2, 5, 15])</code>
--

Benefits:

- Temporary API failures are retried automatically.
- Network instability does not immediately fail the entire pipeline.
- Reduces manual intervention.

Lessons Learned

Database modeling, PostgreSQL schemas, SQLAlchemy ORM, sessions, transactions, Prefect tasks and flows, retries, logging, timezone-aware timestamps, incremental loading and ETL auditing.